

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Bheeshma L	Reg. No.:	192511332
Date:	Aug 19	Duration:	60 minutes

Code of Conduct

I Bheeshma L / 192511332 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bheeshma L

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate

Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments , security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

CO4 AT-2 CI/CD PIPELINE DESIGN DOCUMENT

Intelligent Water Quality Assessment and Contamination Detection Platform

1. Introduction

The **Intelligent Water Quality Assessment and Contamination Detection Platform** is a software system designed to monitor, analyze, and assess water quality using data collected from sensors and other data sources. The platform can process parameters such as pH, temperature, turbidity, dissolved oxygen, electrical conductivity, and other relevant water-quality measurements.

The system analyzes incoming data, identifies abnormal conditions, detects possible contamination, and provides alerts and reports to authorized users. Since the platform may receive continuous data and requires reliable processing, regular software updates, automated testing, and safe deployment are important.

A **Continuous Integration and Continuous Deployment (CI/CD)** pipeline will be designed to automate the software development lifecycle. The pipeline will automatically build the application, execute automated tests, perform code-quality and security checks, package the application, and deploy approved versions to development, staging, and production environments.

The objective of this document is to design a reliable, secure, automated, and maintainable CI/CD pipeline for the Intelligent Water Quality Assessment and Contamination Detection Platform.

2. Objectives of the CI/CD Pipeline

The main objectives are:

1. Automate the software build process.
 2. Automatically validate every code change.
 3. Detect software defects as early as possible.
 4. Automate unit, integration, API, and system testing.
 5. Perform automated code-quality analysis.
 6. Perform security checks on source code and dependencies.
 7. Build and package the application consistently.
 8. Deploy the application automatically to different environments.
 9. Reduce manual deployment errors.
 10. Provide deployment notifications to the development team.
 11. Support rollback when a deployment fails.
 12. Maintain version history of application releases.
 13. Improve development and delivery speed.
 14. Ensure that only tested and approved software reaches production.
-

3. Project Requirements Analysis

The platform consists of several major components:

- User interface/dashboard
- Backend application/API
- Water-quality data processing module
- Contamination detection module
- Database
- Sensor/data ingestion service
- Alert and notification service
- Authentication and authorization module
- Reporting module

Because these components may be updated frequently, the CI/CD pipeline should automatically validate changes before deployment.

CI/CD Needs

The project requires:

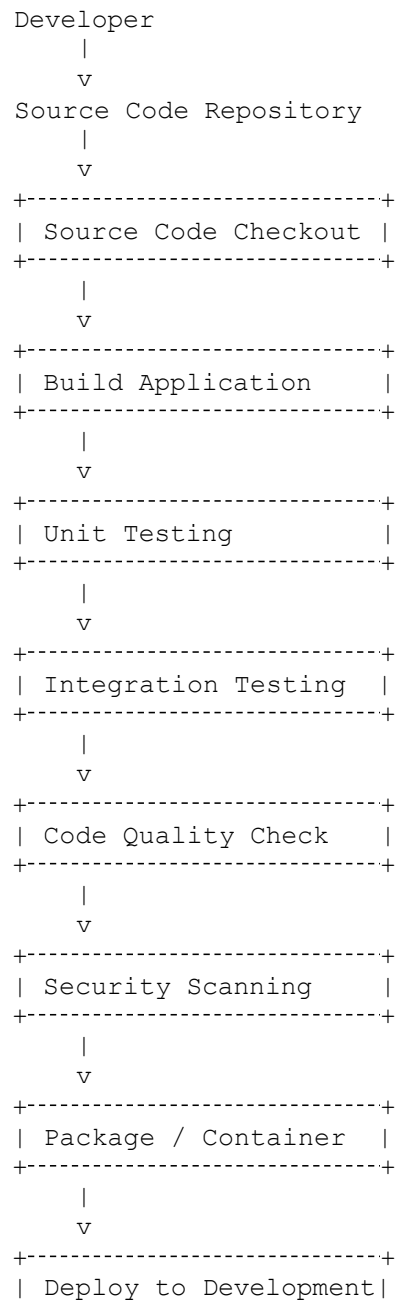
- Source code version control
 - Automated code integration
 - Automated builds
 - Automated testing
 - Code-quality analysis
 - Dependency security scanning
 - Container/image scanning where applicable
 - Automated deployment
 - Environment configuration
 - Monitoring
 - Notifications
 - Rollback capability
-

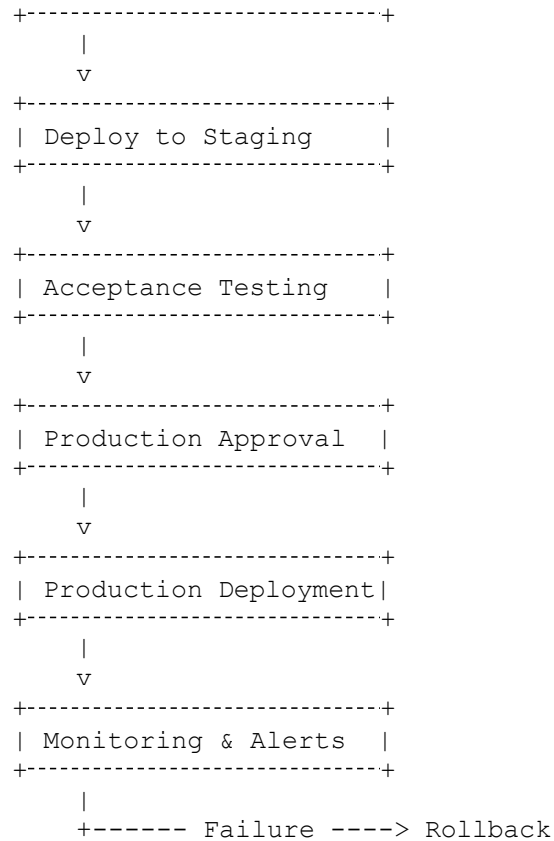
4. Proposed CI/CD Architecture

The proposed pipeline consists of the following stages:

Developer → Git Repository → Build → Unit Testing → Integration Testing → Code Quality → Security Scan → Package/Containerize → Staging Deployment → Acceptance Testing → Production Deployment → Monitoring → Rollback if Required

High-Level Workflow





5. CI/CD Pipeline Stages

Stage 1: Source Code Management

All project source code will be stored in a Git-based repository.

The repository may contain:

```
water-quality-platform/
├── frontend/
├── backend/
├── sensor-service/
├── contamination-detection/
├── database/
├── tests/
├── deployment/
├── Dockerfile
├── CI/CD configuration
└── README.md
```


Activities

- Developer creates or modifies code.
- Developer creates a feature branch.
- Changes are committed to the repository.
- A pull/merge request is created.
- Automated pipeline execution begins.

Benefits

- Version control
 - Collaboration
 - Change tracking
 - Code review
 - Ability to restore previous versions
-

6. Branching Strategy

A simple Git branching strategy is proposed.

Main Branch

The `main` branch contains production-ready code.

Development Branch

The `develop` branch contains code under active development.

Feature Branch

Developers create feature branches for new functionality.

Example:

```
feature/contamination-alert  
feature/sensor-monitoring  
feature/water-quality-report
```

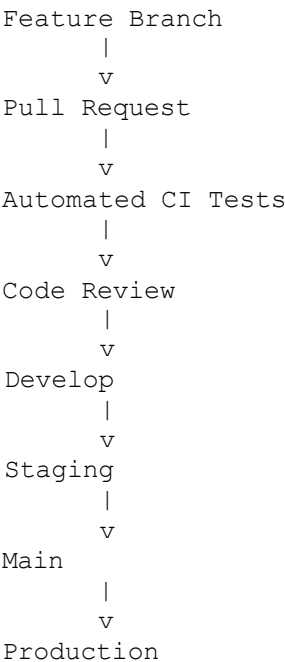
Hotfix Branch

Critical production problems can be fixed using a hotfix branch.

Example:

hotfix/alert-service-failure

Proposed Workflow



7. Tools and Technologies

The following tools are proposed for the CI/CD pipeline.

Pipeline Activity	Proposed Tool	Purpose
Source Control	GitHub	Store and manage source code
CI/CD	GitHub Actions	Automate pipeline
Build	Maven/npm/pip	Build application components
Unit Testing	JUnit/PyTest/Jest	Execute unit tests
API Testing	Postman/Newman	Test APIs
Code Quality	SonarQube	Analyze source code
Security Scanning	OWASP Dependency-Check	Detect vulnerable dependencies
Containerization	Docker	Package application
Container Registry	GitHub Container Registry	Store Docker images
Deployment	Kubernetes/Docker	Deploy application
Infrastructure	Terraform	Infrastructure automation
Monitoring	Prometheus/Grafana	Monitor system health
Notifications	Email/Slack	Notify development team

8. Continuous Integration Workflow

Continuous Integration ensures that every code change is automatically validated before it is merged.

CI Process

```
Code Commit
  |
  v
Checkout Code
  |
  v
Install Dependencies
  |
  v
Build
  |
  v
Unit Tests
  |
  v
Integration Tests
  |
  v
Code Quality Analysis
  |
  v
Security Scan
  |
  v
Build Artifact
```

If any critical stage fails, the pipeline stops and the developer is notified.

9. Build Stage

The build stage compiles and packages the application.

Activities

1. Checkout the latest source code.
2. Install required dependencies.
3. Compile/build the backend.
4. Build the frontend.

5. Build supporting services.
6. Generate application artifacts.
7. Prepare the application for testing.

Expected Result

The application should build successfully without compilation errors.

10. Automated Testing Strategy

Automated testing is an important part of the CI/CD pipeline.

The following testing levels are proposed:

10.1 Unit Testing

Tests individual functions or modules.

Examples:

- Validate pH processing function.
- Test turbidity classification.
- Test contamination scoring.
- Test alert-generation logic.
- Test authentication functions.

10.2 Integration Testing

Tests communication between different components.

Examples:

- Backend ↔ Database
- Sensor service ↔ Backend
- Backend ↔ Alert service
- Backend ↔ Authentication service

10.3 API Testing

Tests backend APIs.

Example:

```
POST /api/sensor/readings
GET /api/water-quality
GET /api/alerts
GET /api/reports
POST /api/login
```

10.4 System Testing

Tests the complete application from the user's perspective.

Example:

```
Sensor Data
  ↓
Backend
  ↓
Analysis
  ↓
Contamination Detection
  ↓
Alert
  ↓
Dashboard
```

10.5 Regression Testing

Existing functionality is tested after every significant change to ensure that new modifications do not break existing features.

10.6 Performance Testing

The system is tested with a large volume of sensor data to determine whether it can maintain acceptable response times.

11. Automated Test Pipeline

The automated testing pipeline can be represented as:

```
Build
|
v
Unit Tests
|
+---- Fail ---> Stop Pipeline
|
v
Integration Tests
|
+---- Fail ---> Stop Pipeline
|
v
API Tests
|
+---- Fail ---> Stop Pipeline
|
v
System/Acceptance Tests
|
+---- Fail ---> Stop Pipeline
|
v
Continue Deployment
```

12. Code Quality Analysis

Code quality analysis will be performed automatically.

The pipeline should check:

- Code duplication
- Code complexity
- Coding standards
- Maintainability
- Potential bugs
- Vulnerabilities
- Test coverage
- Unused code

A quality gate should be configured so that code with serious quality problems does not proceed to production.

Example Quality Rules

```
Unit Test Coverage >= 80%  
Critical Bugs = 0  
Critical Vulnerabilities = 0  
Major Code Smells = Within Approved Limit  
Duplicated Code = Within Approved Limit
```

The exact values should be adjusted according to project requirements.

13. Security Checks

Security should be included throughout the CI/CD pipeline.

Security Testing Includes

1. Dependency vulnerability scanning.
2. Static application security testing.
3. Container image scanning.
4. Secret detection.
5. Authentication testing.
6. Authorization testing.
7. API security testing.

Sensitive Information

Passwords, database credentials, API keys, and tokens should never be stored directly in source code.

Instead, secure environment variables or a secrets-management system should be used.

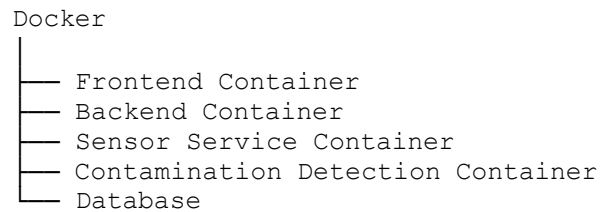
Example:

```
DATABASE_PASSWORD = SECRET  
API_KEY = SECRET  
JWT_SECRET = SECRET
```

14. Docker Containerization

The application can be packaged using Docker to ensure consistency between development, testing, and production environments.

Example Architecture



Benefits

- Consistent environment
 - Easy deployment
 - Portability
 - Isolation
 - Easier scaling
 - Easier rollback
-

15. Container Image Management

After a successful build, a Docker image will be created.

Example:

```
water-quality-platform:v1.0.0
```

The image can be stored in a container registry.

Image Workflow

```
Source Code
  |
  v
Build
  |
  v
Docker Image
  |
  v
Security Scan
  |
  v
Container Registry
  |
  v
Deployment
```

16. Deployment Environments

Three main environments are proposed.

16.1 Development Environment

Used by developers to test new features.

Purpose:
Development and debugging

Changes can be deployed automatically from the development branch.

16.2 Staging Environment

Used for final validation before production.

The staging environment should closely resemble production.

Testing includes:

- Integration testing
 - System testing
 - Acceptance testing
 - Performance testing
 - Security validation
-

16.3 Production Environment

The production environment is used by actual users and monitoring operations.

Only successfully tested and approved releases should be deployed.

17. Deployment Strategy

A controlled deployment strategy will be used.

Deployment Workflow

```
Development
  |
  v
Automated Tests
  |
  v
Staging
  |
  v
Acceptance Tests
  |
  v
Approval
  |
  v
Production
```

18. Deployment Approaches

18.1 Rolling Deployment

New application instances are gradually introduced while old instances continue operating.

Advantages

- Reduced downtime
 - Gradual deployment
 - Easy monitoring
-

18.2 Blue-Green Deployment

Two environments are maintained:

Blue = Current Production
Green = New Version

The new version is deployed to Green and tested. Traffic is switched to Green after successful validation.

If problems occur, traffic can be switched back to Blue.

18.3 Canary Deployment

The new version is initially provided to a small percentage of users or traffic.

For example:

95% → Existing Version
5% → New Version

If monitoring shows no serious problems, traffic can gradually move to the new version.

19. Recommended Deployment Strategy

For this water-quality platform, a **Blue-Green or Canary deployment strategy** is recommended for important production releases.

This is because the platform may provide contamination alerts, and a faulty software release could affect monitoring and alert functionality.

The new version should be validated before all production traffic is transferred.

20. Rollback Mechanism

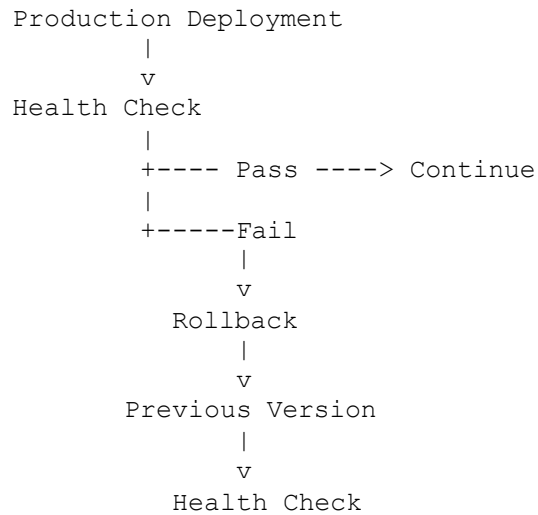
A rollback mechanism is required if a deployment introduces critical problems.

Rollback Conditions

Rollback may occur when:

- Application fails to start.
- Critical tests fail.
- API becomes unavailable.
- Database migration causes serious problems.
- Contamination detection stops functioning.
- Alert notifications fail.
- Performance becomes unacceptable.
- Security vulnerability is discovered.

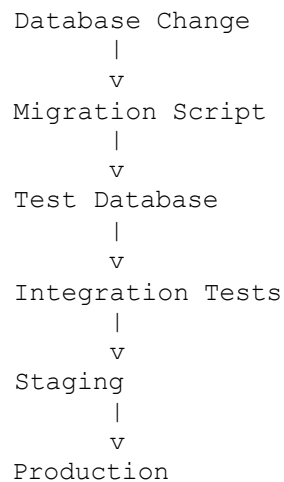
Rollback Workflow



21. Database Migration Strategy

Database changes should also be controlled through the pipeline.

Process



Before production deployment, database backups should be performed where appropriate.

Database changes should be backward-compatible whenever possible to reduce deployment risk.

22. Environment Configuration

Different configuration values should be maintained for each environment.

Configuration	Development	Staging	Production
Database	Dev DB	Staging DB	Production DB
Debug Mode	Enabled	Limited	Disabled
Logging	Detailed	Moderate	Controlled
Alerts	Test	Test	Real
API Keys	Dev secrets	Staging secrets	Production secrets
Monitoring	Basic	Full	Full

Sensitive configuration values should be stored securely and should not be committed to the Git repository.

23. Monitoring and Health Checks

After deployment, the application should be monitored continuously.

Monitoring Areas

- Application availability
- CPU usage
- Memory usage
- Database performance
- API response time
- Error rate
- Sensor data processing
- Alert generation
- Container health
- Deployment health

Health Check Examples

```
GET /health
GET /api/health
GET /api/sensor/status
```

A successful health check indicates that the service is running correctly.

24. Notifications

The CI/CD pipeline should send notifications about important events.

Successful Pipeline

```
Build Successful
Tests Passed
Security Scan Passed
Deployment Successful
```

Failed Pipeline

```
Build Failed
Test Failed
Security Issue Detected
Deployment Failed
```

Notifications can be sent through:

- Email
- Slack
- Microsoft Teams
- Other supported notification systems

25. Quality Gates

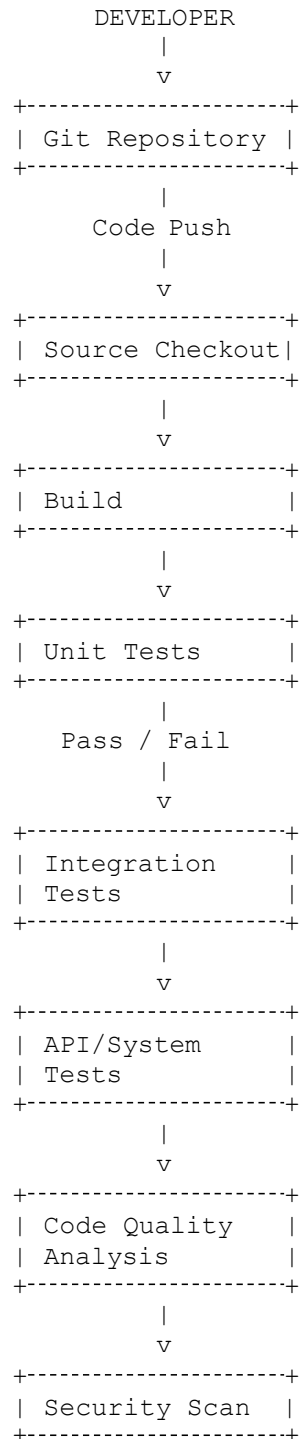
The pipeline should prevent deployment when important quality requirements are not satisfied.

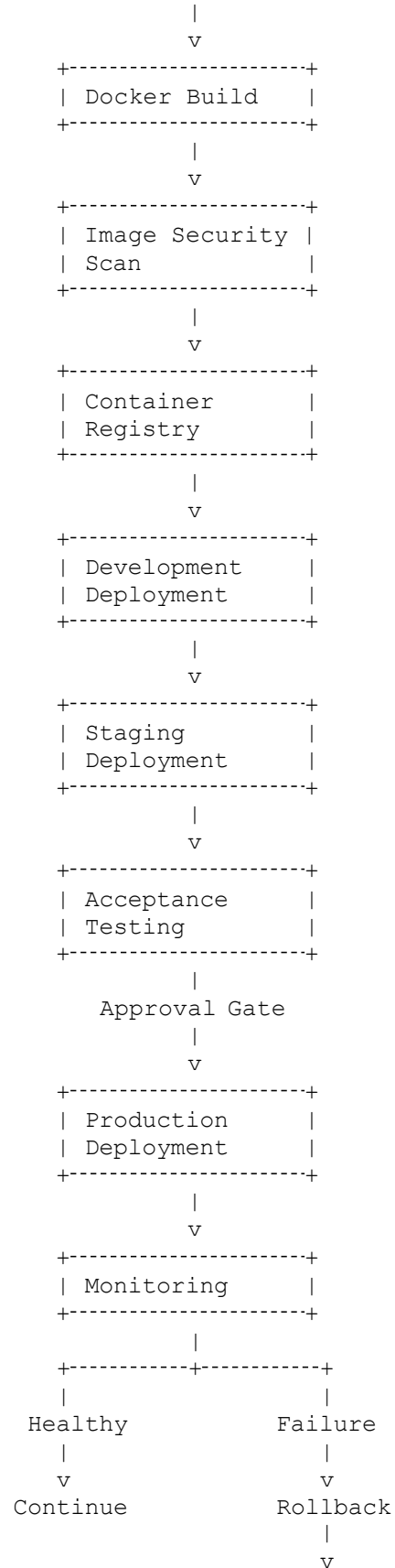
Proposed Quality Gates

Quality Check	Requirement
Build	Must pass
Unit Tests	Must pass
Integration Tests	Must pass
API Tests	Must pass
Critical Bugs	0
Critical Security Vulnerabilities	0
Code Quality	Approved
Test Coverage	Meets project target
Container Scan	No unacceptable critical vulnerabilities

26. CI/CD Pipeline Diagram

The complete pipeline is:





27. Example GitHub Actions Pipeline

A simplified CI/CD workflow can be represented as follows:

```
name: Water Quality Platform CI/CD
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - develop
```

```
      - main
```

```
  pull_request:
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout source code
```

```
        uses: actions/checkout@v4
```

```
      - name: Install dependencies
```

```
        run: |
```

```
          npm install
```

```
      - name: Build application
```

```
        run: |
```

```
          npm run build
```

```
  test:
```

```
    needs: build
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout source code
```

```
        uses: actions/checkout@v4
```

```
      - name: Install dependencies
```

```
        run: |
```

```
          npm install
```

```
      - name: Run unit tests
```

```
        run: |
```

```
          npm test
```

```
  security:
```

```
    needs: test
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
- name: Checkout source code
  uses: actions/checkout@v4

- name: Security scan
  run: |
    npm audit

docker:
  needs: security
  runs-on: ubuntu-latest

steps:
  - name: Checkout source code
    uses: actions/checkout@v4

  - name: Build Docker image
    run: |
      docker build -t water-quality-platform:latest .

deploy:
  needs: docker
  runs-on: ubuntu-latest

steps:
  - name: Deploy application
    run: |
      echo "Deploying application..."
```

This is a conceptual example. The actual pipeline should be modified according to the project's programming language, hosting platform, database, and deployment architecture.

28. CI/CD Security Strategy

Security should be implemented throughout the pipeline rather than only before production deployment.

Security Controls

1. Protected branches.
 2. Mandatory code review.
 3. Secret management.
 4. Dependency scanning.
 5. Static code analysis.
 6. Container image scanning.
 7. Least-privilege deployment credentials.
 8. Authentication and authorization testing
-

29. Failure Handling

The pipeline should stop automatically when a critical stage fails.

Example

```
Code Commit
  |
  v
Build
  |
  FAIL
  |
  v
Pipeline Stops
  |
  v
Developer Notification
  |
  v
Developer Fixes Code
  |
  v
New Commit
  |
  v
Pipeline Runs Again
```

This prevents defective software from reaching later stages.

30. CI/CD Test Strategy for Water Quality Platform

The following tests should be automatically executed:

Test Type	Example
Unit Test	pH classification
Unit Test	Contamination score calculation
Integration Test	Sensor-to-backend communication
API Test	Sensor data submission
API Test	Alert retrieval
Database Test	Reading storage
System Test	Sensor-to-dashboard workflow
Security Test	Unauthorized API access
Regression Test	Existing monitoring functions
Performance Test	High-volume sensor readings
Acceptance Test	Complete user workflow

31. Release Versioning

The project should use version numbers for releases.

Example:

```
v1.0.0  
v1.1.0  
v1.2.0  
v2.0.0
```

Versioning makes it easier to identify and restore previous releases.

Example

```
v1.0.0 → Initial Release  
v1.1.0 → Added Contamination Alerts  
v1.2.0 → Added Historical Reports  
v2.0.0 → Major Platform Update
```

32. Deployment Approval Process

For development:

Automatic Deployment

For staging:

Automatic Deployment + Automated Testing

For production:

Automated Quality Checks + Approval + Deployment

Example:

```
Developer Commit
  |
  v
CI Pipeline
  |
  v
All Tests Passed
  |
  v
Staging
  |
  v
Acceptance Tests
  |
  v
Production Approval
  |
  v
Production
```

33. Backup and Recovery

The platform should maintain appropriate backups of important production data.

Important data includes:

- Water-quality readings
- Sensor configurations
- User information
- Alert history
- Reports
- System configuration

Before major database changes or high-risk deployments, a backup should be created according to the project's operational policy.

34. Advantages of the Proposed CI/CD Pipeline

The proposed pipeline provides the following benefits:

1. Faster software delivery.
 2. Reduced manual errors.
 3. Early detection of defects.
 4. Automated testing.
 5. Improved code quality.
 6. Improved security.
 7. Consistent deployments.
 8. Easy version management.
 9. Quick rollback.
 10. Better collaboration among developers.
 11. Reduced deployment downtime.
 12. Improved reliability of contamination detection functionality.
-

35. Risks and Mitigation

Risk	Impact	Mitigation
Build failure	High	Automated build validation
Test failure	High	Stop pipeline and notify developer
Security vulnerability	Critical	Automated security scanning
Deployment failure	High	Health checks and rollback
Database migration failure	Critical	Backup and migration testing
Incorrect configuration	High	Environment-specific configuration
Secret exposure	Critical	Secure secret management
High traffic/data volume	Medium/High	Performance and scalability testing
Sensor communication failure	High	Monitoring and error handling

36. Requirement-to-Pipeline Traceability

Project Requirement	CI/CD Support
Reliable water-quality assessment	Automated unit and integration tests

Project Requirement	CI/CD Support
Contamination detection	Automated functional and regression tests
Real-time monitoring	Performance and system testing
Secure user access	Security testing
Sensor integration	Integration/API testing
Alert generation	Automated alert tests
Data storage	Database integration testing
Reporting	System and regression testing
Reliable deployment	Automated deployment and health checks
System recovery	Rollback and backup mechanisms
High availability	Monitoring and deployment strategy
Maintainability	Code quality analysis

37. Pipeline Execution Summary

Stage	Input	Process	Output
Source	Developer code	Checkout	Source files
Build	Source files	Compile/build	Build artifact
Unit Test	Application	Unit tests	Test results
Integration Test	Services	Integration tests	Test results
Quality	Source code	Static analysis	Quality report
Security	Code/dependencies	Security scan	Security report
Package	Application	Docker build	Container image
Development	Image	Deployment	Running application
Staging	Tested image	Deployment	Staging system
Acceptance	Staging system	Acceptance tests	Approval result
Production	Approved image	Deployment	Live application
Monitoring	Production	Health monitoring	System status
Rollback	Failed deployment	Restore version	Previous stable release

38. Expected Outcome

After implementing the proposed CI/CD pipeline, every significant code change to the Intelligent Water Quality Assessment and Contamination Detection Platform will pass through automated validation before reaching production.

The pipeline will:

```
Commit
↓
Build
↓
Test
↓
Quality Check
↓
Security Check
↓
Package
↓
Staging
↓
Acceptance
↓
Approval
↓
Production
↓
Monitoring
↓
Rollback if Required
```

This approach will improve the reliability, security, maintainability, and deployment speed of the water-quality platform.

39. Conclusion

The proposed CI/CD pipeline provides an automated and controlled approach for developing, testing, and deploying the **Intelligent Water Quality Assessment and Contamination Detection Platform**.

The pipeline integrates source-code management, automated builds, unit testing, integration testing, API testing, code-quality analysis, security scanning, containerization, staging, production deployment, monitoring, notifications, and rollback mechanisms.

Automating these activities reduces manual errors and allows developers to identify problems early in the development process. Quality gates ensure that software containing critical defects or security vulnerabilities does not proceed to production.

The use of separate development, staging, and production environments provides additional control over software releases. Automated health checks and rollback mechanisms further improve system reliability.

Therefore, the proposed CI/CD architecture provides a suitable foundation for delivering frequent, secure, reliable, and maintainable updates to the Intelligent Water Quality Assessment and Contamination Detection Platform.